



SWEN 221

A surreal, dreamlike landscape. In the center, a person stands on a small, dark, rocky outcrop that appears to be part of a vast, ethereal sea of clouds. The person is looking out over a vast, colorful sky. The sky is filled with large, billowing clouds in shades of pink, purple, blue, and yellow. A bright, glowing light source, possibly a sun or moon, is visible on the horizon, casting a warm glow. The sky is also filled with stars, a comet streaking across the upper left, and a large, colorful nebula or galaxy in the upper right. The overall atmosphere is one of wonder and exploration.

Marco Servetto
CO258

www.youtube.com/MarcoServetto

SWEN221 Lecture 3

Introduction to Inheritance and Subtyping

Interfaces and Records

- We are going to explore how to support inheritance and subtyping using interfaces and records.
- Then we will see how it was done in older Java with classes and single inheritance.
- Inheritance: enables Code reuse within a class hierarchy
(most often it means less code in our own internal API)
- Subtyping: enables Code reuse for our users
(most often it means less code in the program at large)

overall look without Inheritance



class/record A

StuffOfA
CommonStuff
CommonStuff
CommonStuff
CommonStuff
CommonStuff
CommonStuff
CommonStuff

class/record B

StuffOfB
CommonStuff
CommonStuff
CommonStuff
CommonStuff
CommonStuff
CommonStuff
CommonStuff

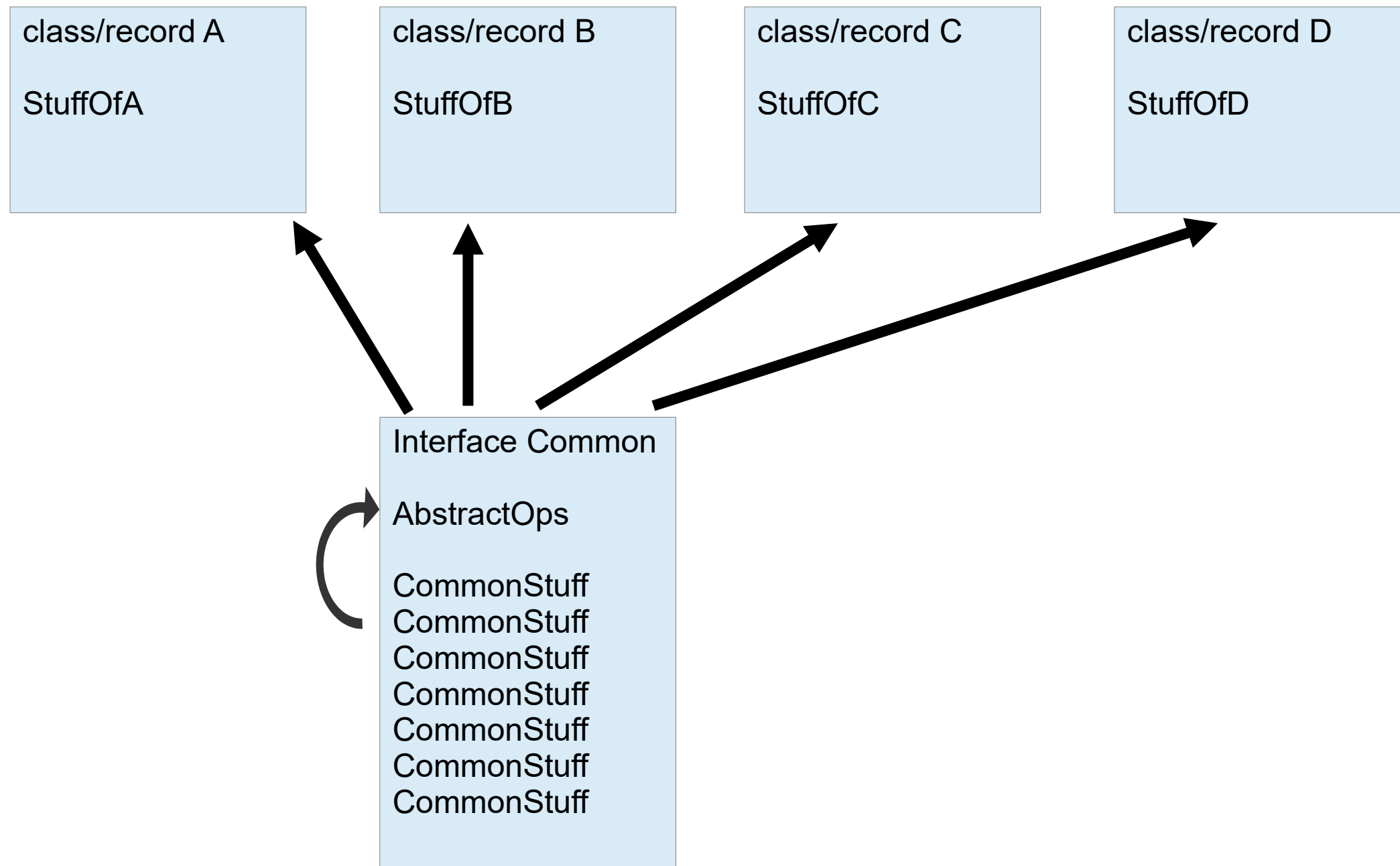
class/record C

StuffOfC
CommonStuff
CommonStuff
CommonStuff
CommonStuff
CommonStuff
CommonStuff
CommonStuff

class/record D

StuffOfD
CommonStuff
CommonStuff
CommonStuff
CommonStuff
CommonStuff
CommonStuff
CommonStuff

overall look with Inheritance



```
record Point2D(double x, double y){}
```

A running horse

```
record Horse(String name, Point2D location){
```

```
    Horse withLocation(Point2D dest){ return new Horse(name,dest); }
```

```
    double speed(){ return 10d; }
```

```
    void draw(Graphics2D g){/*...*/}
```

```
    Horse run(Point2D dest){
```

```
        double s = this.speed(); //Local vars initialized when declared
```

```
        double diffX = dest.x() - location.x();
```

```
        double diffY = dest.y() - location.y();
```

```
        double distance= Math.sqrt(diffX * diffX + diffY * diffY);
```

```
        if (distance < s){ return this.withLocation(dest); }//Early exit (guard) when possible
```

```
        double dir= Math.atan2(diffY,diffX);//Local vars as late as possible
```

```
        return this.withLocation(new Point2D(  
            location.x() + s * Math.cos(dir),//complex subexpressions are fine  
            location.y() + s * Math.sin(dir) //if well formatted  
        ));
```

```
    }
```

```
}
```



```
record Point2D(double x, double y){}
```

A running dog



```
record Dog(String name, Point2D location){
```

```
    Dog withLocation(Point2D dest){ return new Dog(name,dest); }
```

```
    double speed(){ return 5d; }
```

```
    void draw(Graphics2D g){/*...*/}
```

```
    Dog run(Point2D dest){
```

```
        double s = this.speed();
```

```
        double diffX = dest.x() - location.x();
```

```
        double diffY = dest.y() - location.y();
```

```
        double distance= Math.sqrt(diffX * diffX + diffY * diffY);
```

```
        if (distance < s){ return this.withLocation(dest); }
```

```
        double dir= Math.atan2(diffY,diffX);
```

```
        return this.withLocation(new Point2D(
```

```
            location.x() + s * Math.cos(dir),
```

```
            location.y() + s * Math.sin(dir)
```

```
        ));
```

```
    }
```

```
}
```

//Same cutpasted code here.

//Can we reuse the code before somehow?

A running animal



```
interface Animal{ //interface animal: a generalization of dog/horse
    String name();           //fields are now just methods
    Point2D location(); //speed() was a method originally
    double speed();           //note how there is no difference any more!
    void draw(Graphics2D g);   //abstract methods for unique behavior
    Animal withLocation(Point2D dest); //abstract methods for unique behavior
```

```
default Animal run(Point2D dest){ //default methods for common behavior
    double s      = this.speed();
    double diffX   = dest.x() - location().x();
    double diffY   = dest.y() - location().y();
    double distance= Math.sqrt(diffX * diffX + diffY * diffY);
    if (distance < s){ return this.withLocation(dest); }
    double dir= Math.atan2(diffY,diffX);
    return this.withLocation(new Point2D(
        location().x() + s * Math.cos(dir),
        location().y() + s * Math.sin(dir)
    ));
}
```



A running animal

```
record Horse(String name, Point2D location) implements Animal{  
    public Horse withLocation(Point2D dest){ return new Horse(name,dest); }  
    public double speed(){ return 10d; }  
    public void draw(Graphics2D g){/*..*/}  
}
```

```
record Dog(String name, Point2D location) implements Animal{  
    public Dog withLocation(Point2D dest){ return new Dog(name,dest); }  
    public double speed(){ return 5d; }  
    public void draw(Graphics2D g){/*..*/}  
}
```

//Now Horse and Dog code is minimal and without duplication

//Note: fields name and location automatically generate getters
//suitable to satisfy the abstract getters in the interface.
//That is one of the biggest advantages of records+interfaces!



But.. the dog is not moving...

We are just making a new dog that looks like the old one but it is in another location.

- This way of coding is actually quite good at preventing subtitle aliasing mistakes, and given the improvements in java objects allocation speed, it is not a big problem for performance.
- However, what if we DO WANT to actually mutate our dog?



A mutable running animal

```
interface Animal{
    String name();
    Point2D location();
    void location(Point2D location); //new method: a setter!
    double speed();
    void draw(Graphics2D g);

    default void run(Point2D dest){
        double s      = this.speed();
        double diffX   = dest.x() - location().x();
        double diffY   = dest.y() - location().y();
        double distance= Math.sqrt(diffX * diffX + diffY * diffY);
        if (distance < s){ this.location(dest); return; } //acceptable in 1 line
        double dir = Math.atan2(diffY,diffX);
        this.location(new Point2D(
            location().x() + s * Math.cos(dir),
            location().y() + s * Math.sin(dir)
        ));
    }
}
```

A mutable running animal

```
class Horse implements Animal{
    public Horse(String name, Point2D location){ this.name= name; this.location= location; }
    private String name; private Point2D location;
    public String name(){ return name; }
    public Point2D location(){ return location; }
    public void location(Point2D location){ this.location = location; }

    public double speed(){ return 10d; }
    public void draw(Graphics2D g){/*..*/}
}
```

```
class Dog implements Animal{
    public Dog(String name, Point2D location){ this.name= name; this.location= location; }
    private String name; private Point2D location;
    public String name(){ return name; }
    public Point2D location(){ return location; }
    public void location(Point2D location){ this.location = location; }

    public double speed(){ return 5d; }
    public void draw(Graphics2D g){/*..*/}
}
```

//The interface is about the same size, but the classes are bigger then the records
//Also: I've not declared toString, equals and hashCode.

A mutable running animal

```
class Horse implements Animal{
    public Horse(String name, Point2D location){ this.name= name; this.location= location; }
    private String name; private Point2D location;
    public String name(){ return name; }
    public Point2D location(){ return location; }
    public void location(Point2D location){ this.location = location; }

    public double speed(){ return 10d; }
    public void draw(Graphics2D g){/*..*/}
}
```

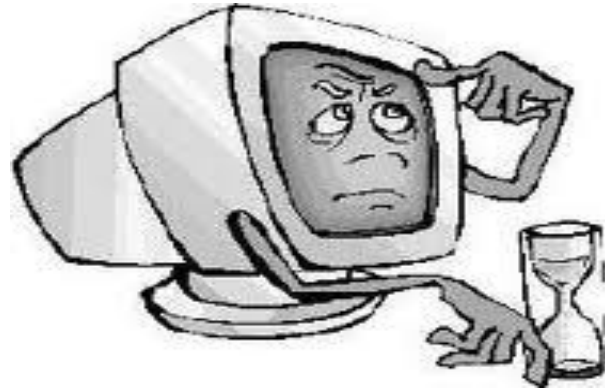
```
class Dog implements Animal{
    public Dog(String name, Point2D location){ this.name= name; this.location= location; }
    private String name; private Point2D location;
    public String name(){ return name; }
    public Point2D location(){ return location; }
    public void location(Point2D location){ this.location = location; }

    public double speed(){ return 5d; }
    public void draw(Graphics2D g){/*..*/}
}
```

//The interface is about the same size, but the classes are bigger then the records
//Also: I've not declared toString, equals and hashCode.

```
Animal x= .....
```

```
x.run(myDest);
```



```
class Horse implements Animal{
```

```
..
```

```
public double speed(){
```

```
    return 10d;
```

```
}
```

```
..
```

```
}
```

```
interface Animal{
```

```
..
```

```
default void run(Point2D dest){
```

```
    double s= this.speed();
```

```
    ..
```

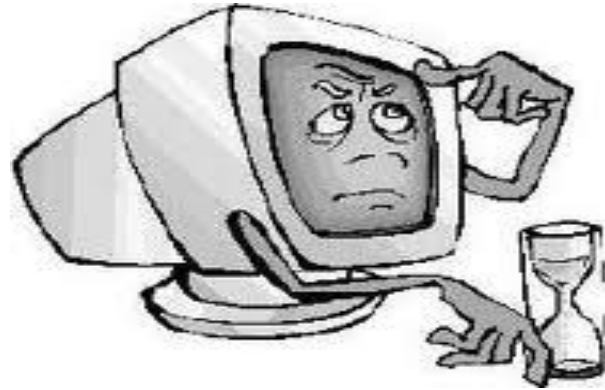
```
}
```

```
}
```



```
Animal x= ....
```

```
x.run(myDest);
```



```
Animal x= new Horse(...);
```

```
x.run(myDest);
```



```
class Horse implements Animal{
```

```
..
```

```
public double speed(){
```

```
    return 10d;
```

```
}
```

```
..
```

```
}
```

```
interface Animal{
```

```
..
```

```
default void run(Point2D dest){
```

```
    double s= this.speed();
```

```
    ..
```

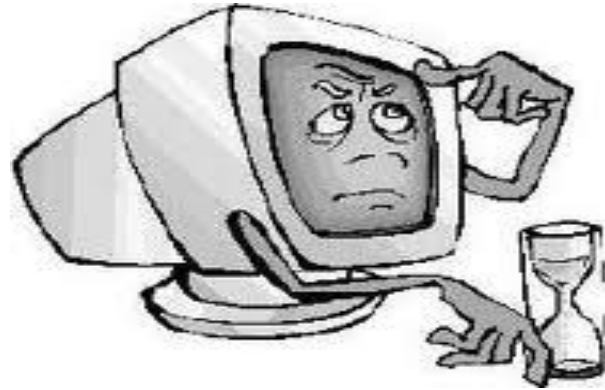
```
}
```

```
}
```



```
Animal x= ....
```

```
x.run(myDest);
```



```
Animal x= new Horse(...);
```

```
x.run(myDest);
```



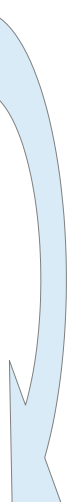
```
Animal x= new Horse(...);
```

```
x.run(myDest); <-- Animal.run  
... double s= this.speed();
```



```
class Horse implements Animal{  
    ..  
    public double speed(){  
        return 10d;  
    }  
    ..  
}
```

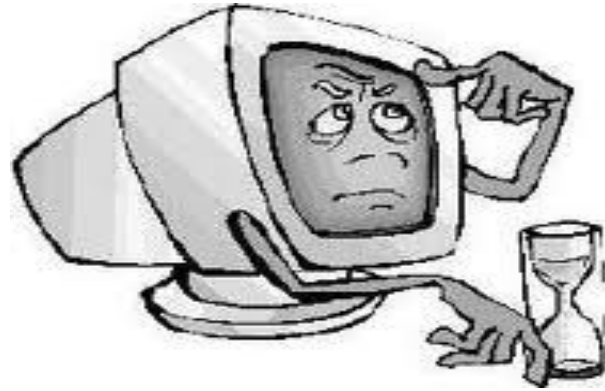
```
interface Animal{  
    ..  
    default void run(Point2D dest){  
        double s= this.speed();  
        ..  
    }  
}
```



```
Animal x= ....  
  
x.run(myDest);  
    ... double s= this.speed();
```

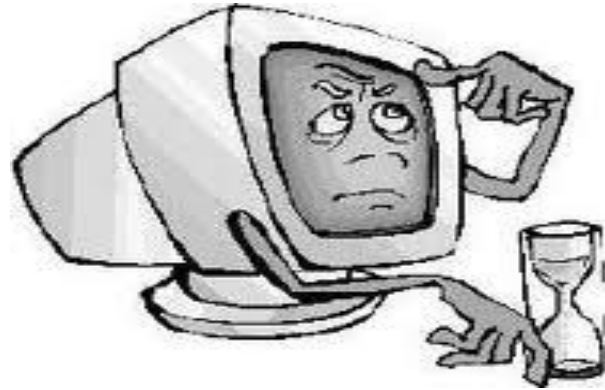
```
class Horse implements Animal{  
    ..  
    public double speed(){  
        return 10d;  
    }  
    ..  
}  
  
interface Animal{  
    ..  
    default void run(Point2D dest){  
        double s= this.speed();  
        ..  
    }  
}
```

```
Animal x= ....  
x.run(myDest);  
... double s= this.speed();
```



```
class Horse implements Animal{  
    ..  
    public double speed(){  
        return 10d;  
    }  
    ..  
}  
  
interface Animal{  
    ..  
    default void run(Point2D dest){  
        double s= this.speed();  
        ..  
    }  
}
```

```
Animal x= ....  
x.run(myDest);  
... double s= this.speed();
```

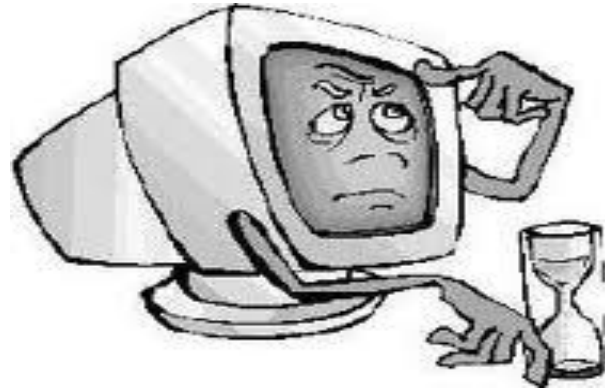


```
Animal x= new Horse(...);  
x.run(myDest);  
... double s= this.speed();
```



```
class Horse implements Animal{  
    ..  
    public double speed(){  
        return 10d;  
    }  
    ..  
}  
  
interface Animal{  
    ..  
    default void run(Point2D dest){  
        double s= this.speed();  
        ..  
    }  
}
```

```
Animal x= ....  
x.run(myDest);  
... double s= this.speed();
```



```
Animal x= new Horse(...);  
x.run(myDest);  
... double s= this.speed();
```



```
Animal x= new Horse(...);  
x.run(myDest);  
... double s= 10;
```



```
class Horse implements Animal{  
    ..  
    public double speed(){ ←  
        return 10d;  
    }  
    ..  
}  
  
interface Animal{  
    ..  
    default void run(Point2D dest){  
        double s= this.speed();  
        ..  
    }  
}
```

Overriding

- Defining a new implementation for an existing method.
- That is,

One method → many different implementations

Two mental models

- Look up: (as shown in the example)
At run time, the computer explores the hierarchy and searches for the most specific implementation.
- Flattening:
At compile time, the methods are imported from the base (Animal) into the heir (Horse and Dog). Then the runtime simply takes the methods from the code of the current class.
- Both are good mental model. Chose the one you prefer

Quiz 1: what happens here?

A > B > C

What does it print?

RULE:

**It explores the hierarchy
and searches for
the most specific
implementation.**

```
interface A{  
    default String m() {return "A";}  
}  
interface B extends A{  
    default String m() {return "B";}  
}  
class C implements B{  
    public String m() {return "C";}  
}
```

```
System.out.println(new C().m());
```

Quiz 1: what happens here?

A > B > C

What does it print?

RULE:

**It explores the hierarchy
and searches for
the most specific
implementation.**

```
interface A{  
    default String m() {return "A";}  
}  
interface B extends A{  
    default String m() {return "B";}  
}  
class C implements B{  
    public String m() {return "C";}  
}
```

```
System.out.println(new C().m());
```

“C”

- Method is defined directly in C, so A.m and B.m are ignored
C.m is selected

Quiz 2: what happens here?

A > B > C

What does it print?

RULE:

**It explores the hierarchy
and searches for
the most specific
implementation.**

```
interface A{  
    default String m() {return "A";}  
}  
interface B extends A{  
    default String m() {return "B";}  
}  
class C implements B{  
  
}
```

```
System.out.println(new C().m());
```

Quiz 2: what happens here?

A > B > C

What does it print?

RULE:

It explores the hierarchy and searches for the most specific implementation.

```
interface A{  
    default String m() {return "A";}  
}  
interface B extends A{  
    default String m() {return "B";}  
}  
class C implements B{  
  
}
```

```
System.out.println(new C().m());
```

“B”

- Method is defined in B and A, but B is more specific than A
B.m is selected

Quiz 3: what happens here?

A > B > C

What does it print?

RULE:

**It explores the hierarchy
and searches for
the most specific
implementation.**

```
interface A{  
    default String m() {return "A";}  
}  
interface B extends A{  
    default String m() {return "B";}  
}  
class C implements A,B{  
  
}
```

```
System.out.println(new C().m());
```

Quiz 3: what happens here?

A > B > C

What does it print?

RULE:

It explores the hierarchy and searches for the most specific implementation.

```
interface A{  
    default String m() {return "A";}  
}  
interface B extends A{  
    default String m() {return "B";}  
}  
class C implements A,B{  
  
}
```

```
System.out.println(new C().m());
```

“B”

- B is still more specific than A. B.m is selected.
This would change in other languages (python)

Quiz 4: what happens here?

A > B > C

What does it print?

RULE:

```
interface A{  
    default String m() {return "A";}  
}  
interface B extends A{  
  
}  
class C implements B{  
  
}
```

```
System.out.println(new C().m());
```

**It explores the hierarchy
and searches for
the most specific
implementation.**

Quiz 4: what happens here?

A > B > C

What does it print?

RULE:

It explores the hierarchy and searches for the most specific implementation.

```
interface A{  
    default String m() {return "A";}  
}  
interface B extends A{  
  
}  
class C implements B{  
  
}
```

```
System.out.println(new C().m());
```

“A”

- Method is only defined in A, so A.m is selected

Quiz 5: what happens here?

A > B, A > C

What does it print?

RULE:

It explores the hierarchy and searches for the most specific implementation.

```
interface A{  
    default String m() {return "A";}  
}  
interface B extends A{  
    default String m() {return "B";}  
}  
class C implements A{  
  
}
```

```
System.out.println(new C().m());
```

Quiz 5: what happens here?

A > B, A > C

What does it print?

RULE:

It explores the hierarchy and searches for the most specific implementation.

```
interface A{  
    default String m() {return "A";}  
}  
interface B extends A{  
    default String m() {return "B";}  
}  
class C implements A{  
  
}
```

```
System.out.println(new C().m());
```

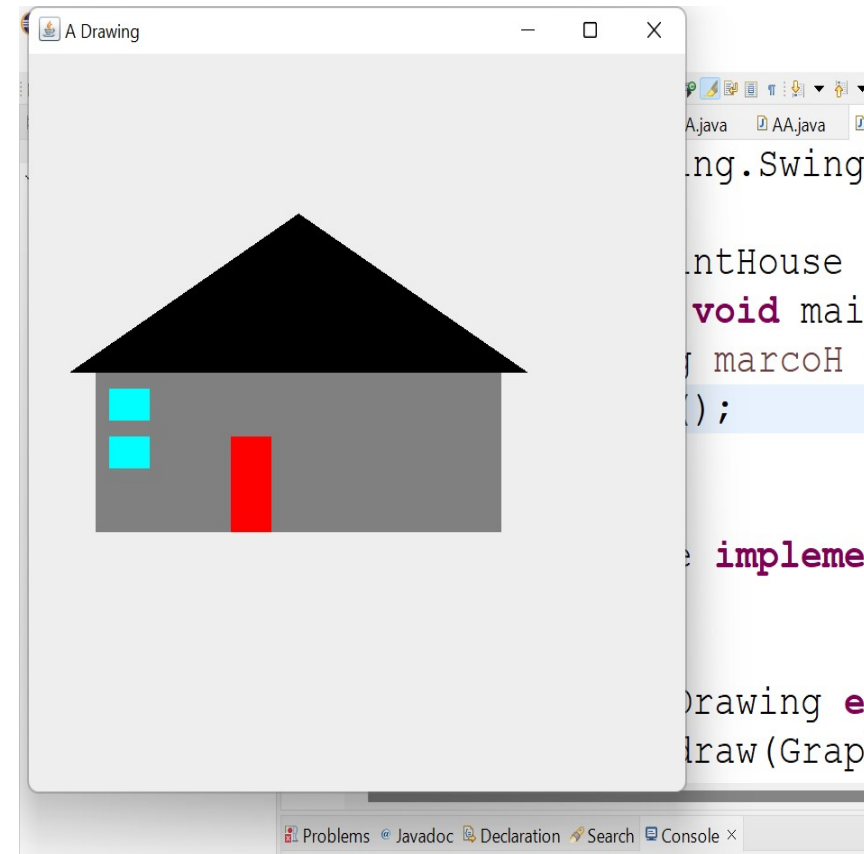
“A”

- C only implements A, thus A.m is selected

Code reuse example

```
public class PaintHouse {  
    public static void main(String[] arg){  
        HouseDrawing marcoH= new MarcoHouse();  
        marcoH.show();  
    }  
}  
class MarcoHouse implements HouseDrawing{  
    //EMPTY!  
}  
interface HouseDrawing extends Drawing{  
    ..  
}
```

- How are we getting a house if the 'MarcoHouse' class is empty??
- By inheriting the behavior of HouseDrawing!



```

interface HouseDrawing extends Drawing{
    default void draw(Graphics2D g){
        drawWall(g);
        drawWindows(g);
        drawDoor(g);
        drawRoof(g);
    }
    default void drawWall(Graphics2D g){
        g.setColor(Color.GRAY);
        g.fillRect(50,200,300,100);
    }
    default void drawWindows(Graphics2D g){
        g.setColor(Color.CYAN);
        g.fillRect(60,210,30,20);
        g.fillRect(60,240,30,20);
    }
    default void drawDoor(Graphics2D g){
        g.setColor(Color.RED);
        g.fillRect(150,240,30,60);
    }
    default void drawRoof(Graphics2D g){
        g.setColor(Color.BLACK);
        g.fillPolygon(new int[]{30,200,370},
                     new int[]{200,100,200},3);
    }
}

```

Code reuse example

Method HouseDrawing.draw

it calls many sub methods with descriptive name.

Why it is good?

We could just have a single method, right?


```
interface HouseDrawing extends Drawing{  
    default void draw(Graphics2D g){  
        //draw wall  
        g.setColor(Color.GRAY);  
        g.fillRect(50,200,300,100);  
  
        //draw windows  
        g.setColor(Color.CYAN);  
        g.fillRect(60,210,30,20);  
        g.fillRect(60,240,30,20);  
  
        //draw door  
        g.setColor(Color.RED);  
        g.fillRect(150,240,30,60);  
  
        //draw roof  
        g.setColor(Color.BLACK);  
        g.fillPolygon(new int[] {30,200,370},  
                    new int[] {200,100,200},3);  
    }  
}
```

Code reuse example

We could just have a single method, right?

Ok, now with comments...

Now it is near as long as the original code, and it is still worst.

Sub method with good name
>
comments for code sections

```

interface HouseDrawing extends Drawing{
    default void draw(Graphics2D g){
        drawWall(g);
        drawWindows(g);
        drawDoor(g);
        drawRoof(g);
    }
    default void drawWall(Graphics2D g){
        g.setColor(Color.GRAY);
        g.fillRect(50,200,300,100);
    }
    default void drawWindows(Graphics2D g){
        g.setColor(Color.CYAN);
        g.fillRect(60,210,30,20);
        g.fillRect(60,240,30,20);
    }
    default void drawDoor(Graphics2D g){
        g.setColor(Color.RED);
        g.fillRect(150,240,30,60);
    }
    default void drawRoof(Graphics2D g){
        g.setColor(Color.BLACK);
        g.fillPolygon(new int[]{30,200,370},
                     new int[]{200,100,200},3);
    }
}

```

Code reuse example

Method HouseDrawing.draw

So, why it is so good in this form?

What are those methods good for?

```

interface HouseDrawing extends Drawing{
    default void draw(Graphics2D g){
        drawWall(g);
        drawWindows(g);
        drawDoor(g);
        drawRoof(g);
    }
    default void drawWall(Graphics2D g){
        g.setColor(Color.GRAY);
        g.fillRect(50,200,300,100);
    }
    default void drawWindows(Graphics2D g){
        g.setColor(Color.CYAN);
        g.fillRect(60,210,30,20);
        g.fillRect(60,240,30,20);
    }
    default void drawDoor(Graphics2D g){
        g.setColor(Color.RED);
        g.fillRect(150,240,30,60);
    }
    default void drawRoof(Graphics2D g){
        g.setColor(Color.BLACK);
        g.fillPolygon(new int[]{30,200,370},
                     new int[]{200,100,200},3);
    }
}

```

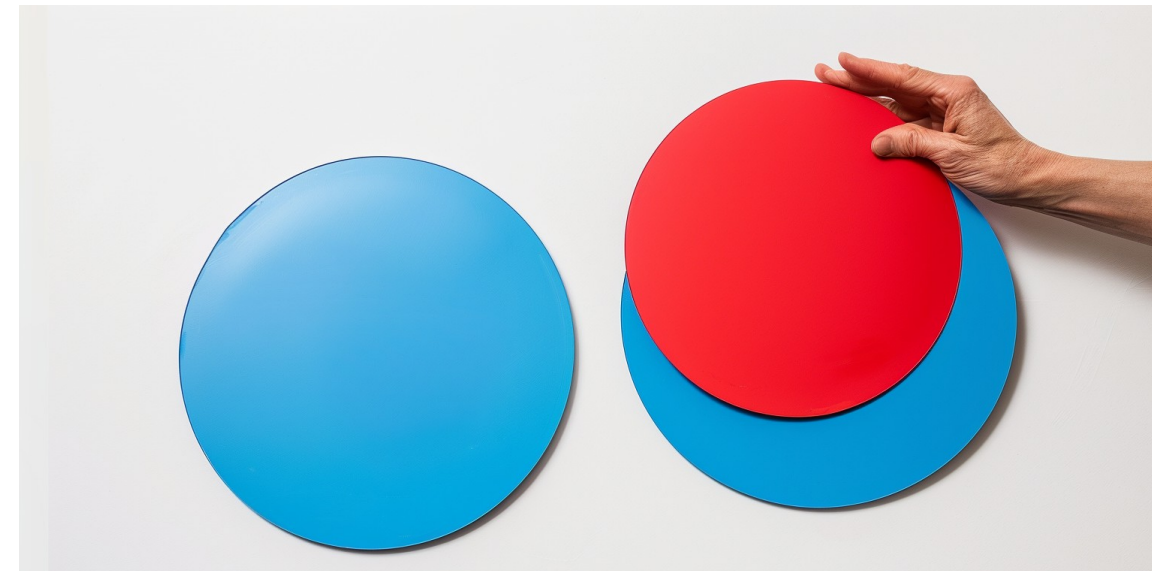
Code reuse example

Method HouseDrawing.draw

So, why it is so good in this form?

What are those methods good for?

OVERRIDING!




```

public class PaintHouse {
    public static void main(String[] arg){
        HouseDrawing marcoH= new MarcoHouse();
        marcoH.show();
    }
}

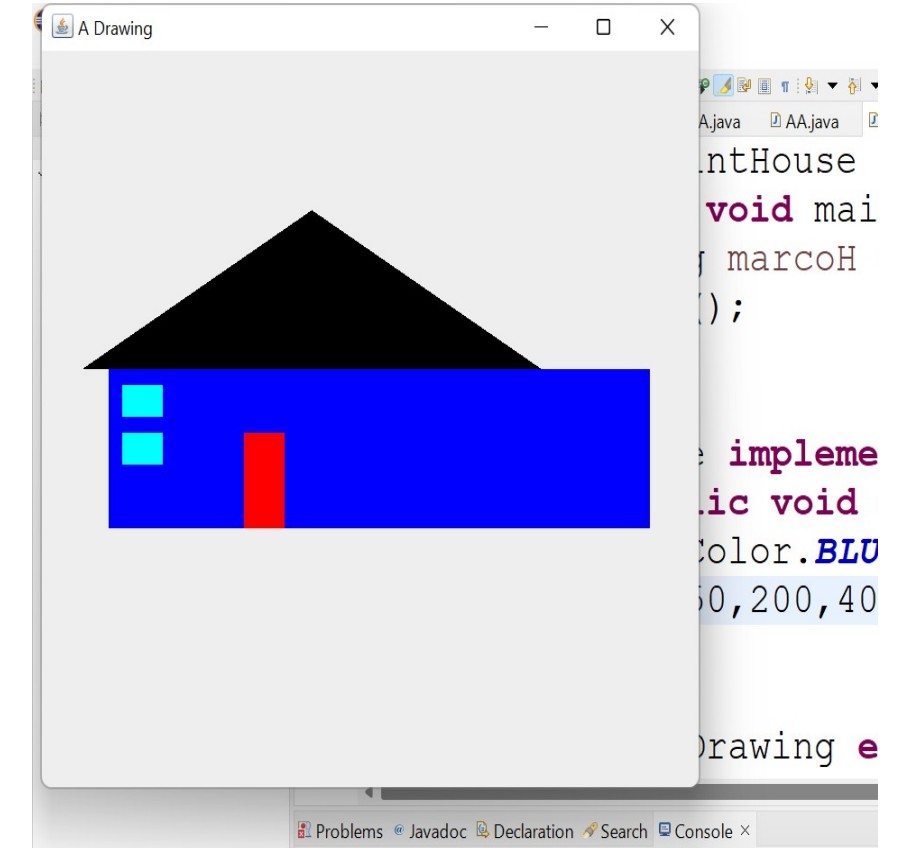
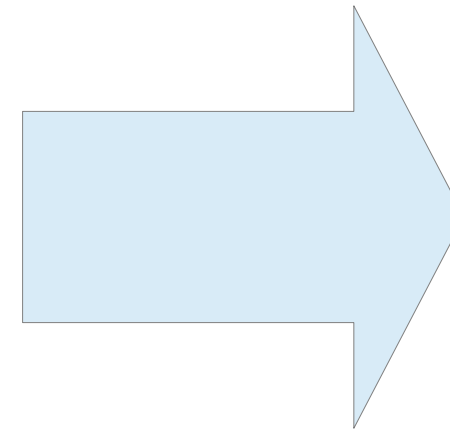
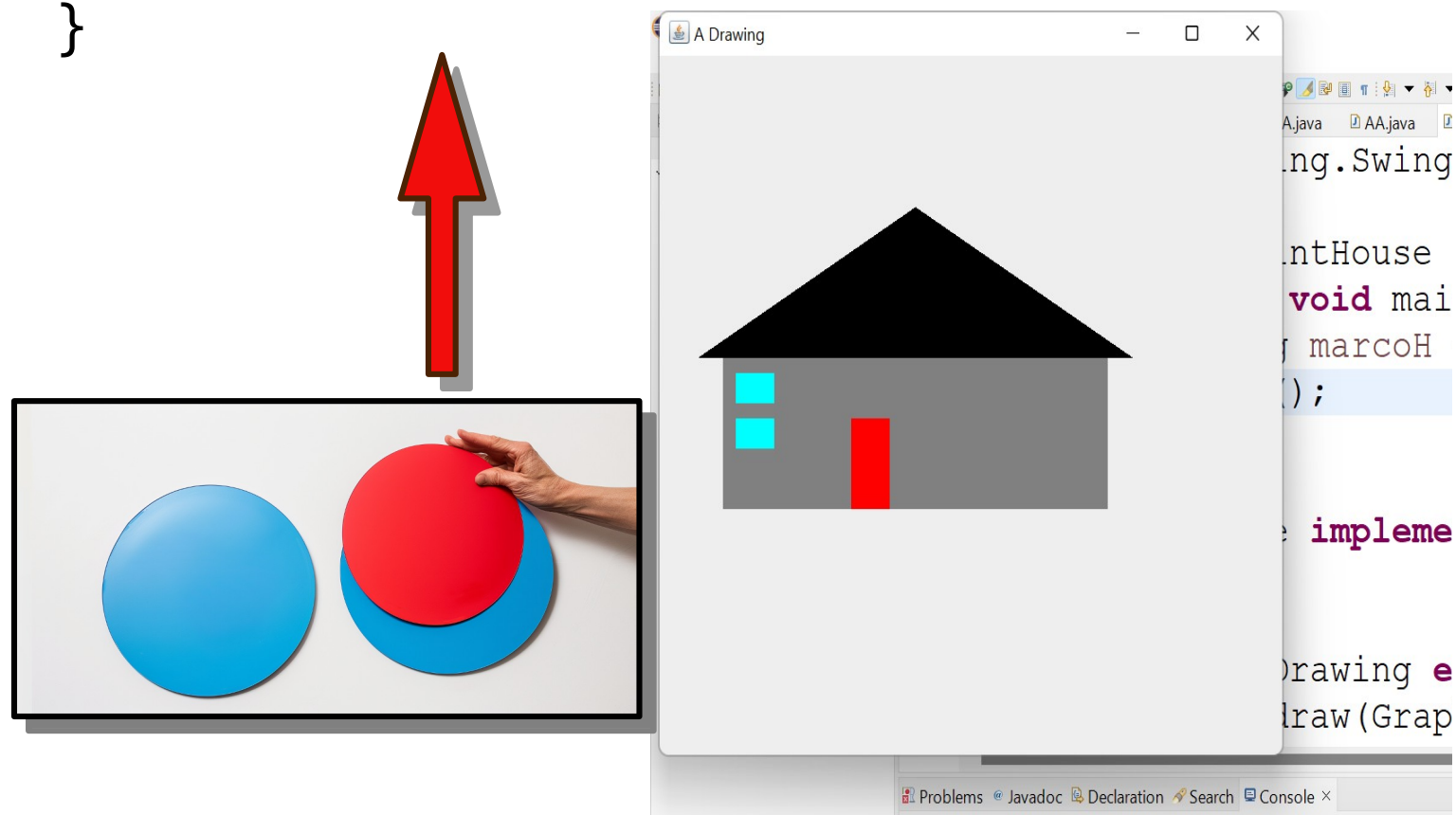
class MarcoHouse implements HouseDrawing{
    @Override
    public void drawWall(Graphics2D g){
        g.setColor(Color.BLUE);
        g.fillRect(50,200,400,100);
    }
}

```

Code reuse example

- By Overriding just the behavior we need, we can specify the changes we want.

No need to repeat all the code.



Template method pattern

- This way of programming is called “The template method pattern”
- PaintHouse.draw is the **template method**
- The methods called by PaintHouse.draw are **the hook methods**.
(PaintHouse.drawWall, PaintHouse.drawWindow ..)

```
//At this stage, you can ignore this code
interface Drawing{//It is just to connect with Swing
    void draw(Graphics2D g);
    default void show(){ SwingUtilities.invokeLater(this::_show); }
    default void _show(){
        JFrame f=new JFrame("A Drawing");
        f.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        f.setPreferredSize(size());
        f.add(new JPanel(){//Anonymous inner class, will see later
            private static final long serialVersionUID= 1L;
            @Override protected void paintComponent(Graphics g){
                super.paintComponent(g); //super call, will see later
                draw((Graphics2D)g);
            }});
        f.pack();
        f.setVisible(true);
    }
    default Dimension size(){ return new Dimension(500,500); }
}
```

The rest of the code

- If you curious, here is the rest of the code needed to draw the house.
- A 'Drawing' interface allowing to draw stuff using a 'show' method.

See you on Monday!

- Lecture 4:
Time travel!

